SelVeri: Self-Verifying Programming Language
High Level Language Syntax and Semantics Document
v0.1.1

Yusuf Demir
Yağız Kaan Aydoğdu

March 1, 2026

1 Syntax

1.1 Lexical Categories

Identities (names) of objects must start with a letter or underscore, and continue with letters, digits or undercases. Literals are integers or floating point numbers as SelVeri supports just these types.

$$\begin{array}{rcl} \langle \mathit{Ident} \rangle & ::= & \text{[A-Za-z\_][A-Za-z0-9\_]}^* \\ \langle \mathit{IntLit} \rangle & ::= & \text{[0-9]}^+ \\ \langle \mathit{FloatLit} \rangle & ::= & \text{[0-9]}^+ \text{.[0-9]}^+ \end{array}$$

1.2 Types

All variables are of types of **Int**, **Float** or **List** of those. Lists contain values of only one type and their size **must** be explicitly stated in the declaration as an arithmetic expression.

$$\langle \mathit{Type} \rangle \quad ::= \quad \texttt{Int} \mid \texttt{Float} \mid \texttt{List}\,[\langle \mathit{Type} \rangle,\ \langle \mathit{AExp} \rangle]$$

1.3 Immediates

$$\langle \mathit{Imm} \rangle \quad ::= \quad \langle \mathit{IntLit} \rangle \mid \langle \mathit{FloatLit} \rangle \mid [\,] \mid [\,\langle \mathit{Imm} \rangle\ (,\ \langle \mathit{Imm} \rangle)^*\,]$$

1.4 Expressions

Arithmetic Expressions (*AExp*)

$$\begin{array}{rcl} \langle \mathit{AExp} \rangle & ::= & \langle \mathit{Imm} \rangle \mid \langle \mathit{Ident} \rangle \mid (\langle \mathit{AExp} \rangle) \\ & & \mid \langle \mathit{AExp} \rangle + \langle \mathit{AExp} \rangle \mid \langle \mathit{AExp} \rangle - \langle \mathit{AExp} \rangle \mid \langle \mathit{AExp} \rangle * \langle \mathit{AExp} \rangle \mid \langle \mathit{AExp} \rangle \,/\, \langle \mathit{AExp} \rangle \\ & & \mid \texttt{len}(\langle \mathit{Ident} \rangle) \mid \langle \mathit{Ident} \rangle [\langle \mathit{AExp} \rangle] \end{array}$$

Boolean Expressions (*BExp*).

$$\begin{array}{rcl} \langle \mathit{BExp} \rangle & ::= & \texttt{true} \mid \texttt{false} \mid \langle \mathit{AExp} \rangle \\ & & \mid \langle \mathit{AExp} \rangle = \langle \mathit{AExp} \rangle \mid \langle \mathit{AExp} \rangle <= \langle \mathit{AExp} \rangle \mid \langle \mathit{AExp} \rangle < \langle \mathit{AExp} \rangle \\ & & \mid \langle \mathit{AExp} \rangle >= \langle \mathit{AExp} \rangle \mid \langle \mathit{AExp} \rangle > \langle \mathit{AExp} \rangle \\ & & \mid \texttt{not}\ \langle \mathit{BExp} \rangle \mid \langle \mathit{BExp} \rangle\ \texttt{and}\ \langle \mathit{BExp} \rangle \mid \langle \mathit{BExp} \rangle\ \texttt{or}\ \langle \mathit{BExp} \rangle \mid \langle \mathit{BExp} \rangle\ \texttt{xor}\ \langle \mathit{BExp} \rangle \mid (\langle \mathit{BExp} \rangle) \end{array}$$

Expressions

$$\langle \mathit{Expr} \rangle \quad ::= \quad \langle \mathit{AExpr} \rangle \mid \langle \mathit{BExpr} \rangle$$

1.5 Specifications (Runtime Assertions)

$$\begin{array}{rcl} \langle \mathit{Spec} \rangle & ::= & \langle \mathit{FOL} \rangle \mid \langle \mathit{PastLTL} \rangle \mid \langle \mathit{FutureLTL} \rangle \\ \langle \mathit{PastLTL} \rangle & ::= & \texttt{Previously}\ \langle \mathit{Spec} \rangle \mid \texttt{Once}\ \langle \mathit{Spec} \rangle \mid \texttt{Historically}\ \langle \mathit{Spec} \rangle \mid \langle \mathit{Spec} \rangle\ \texttt{Since}\ \langle \mathit{Spec} \rangle \\ \langle \mathit{FutureLTL} \rangle & ::= & \texttt{Next}\ \langle \mathit{Spec} \rangle \mid \texttt{Eventually}\ \langle \mathit{Spec} \rangle \mid \texttt{Always}\ \langle \mathit{Spec} \rangle \mid \langle \mathit{Spec} \rangle\ \texttt{Until}\ \langle \mathit{Spec} \rangle \\ \langle \mathit{FOL} \rangle & ::= & \langle \mathit{BExp} \rangle \mid \langle \mathit{FOL} \rangle\ \&\&\ \langle \mathit{FOL} \rangle \mid \langle \mathit{FOL} \rangle\ ||\ \langle \mathit{FOL} \rangle \\ & & \mid !\,\langle \mathit{FOL} \rangle \mid \langle \mathit{FOL} \rangle => \langle \mathit{FOL} \rangle \mid (\langle \mathit{FOL} \rangle) \\ & & \mid \texttt{Forall}\ \langle \mathit{Ident} \rangle\ \langle \mathit{DomainOpt} \rangle\ .\ \langle \mathit{FOL} \rangle \\ & & \mid \texttt{Exists}\ \langle \mathit{Ident} \rangle\ \langle \mathit{DomainOpt} \rangle\ .\ \langle \mathit{FOL} \rangle \\ \langle \mathit{DomainOpt} \rangle & ::= & \epsilon \mid \texttt{in}\ \langle \mathit{Domain} \rangle \\ \langle \mathit{Domain} \rangle & ::= & \langle \mathit{Interval} \rangle \mid \langle \mathit{Ident} \rangle \mid \langle \mathit{SetLiteral} \rangle \mid \langle \mathit{Type} \rangle \\ \langle \mathit{Interval} \rangle & ::= & [\langle \mathit{AExp} \rangle, \langle \mathit{AExp} \rangle) \mid [\langle \mathit{AExp} \rangle, \langle \mathit{AExp} \rangle] \\ \langle \mathit{SetLiteral} \rangle & ::= & \{\langle \mathit{AExpList} \rangle\} \\ \langle \mathit{AExpList} \rangle & ::= & \langle \mathit{AExp} \rangle \mid \langle \mathit{AExp} \rangle, \langle \mathit{AExpList} \rangle \end{array}$$

1.6 Statements and Programs

Statements

$$
\begin{array}{rcl}
\langle Decl \rangle & ::= & \langle Ident \rangle : \langle Type \rangle \\
\langle Assign \rangle & ::= & \langle Ident \rangle := \langle Expr \rangle \\
\langle ListAssign \rangle & ::= & \langle Ident \rangle \, [\langle AExpr \rangle] := \langle Expr \rangle \\
\langle Stmt \rangle & ::= & \langle Decl \rangle \mid \langle Assign \rangle \mid \langle ListAssign \rangle \mid \texttt{noop} \\
 & & \mid \langle Stmt \rangle \, ; \langle Stmt \rangle \\
 & & \mid \texttt{if} \, \langle BExp \rangle \, \texttt{then} \, \langle Stmt \rangle \, [\texttt{else} \, \langle Stmt \rangle] \, \texttt{fi} \\
 & & \mid \texttt{while} \, \langle BExp \rangle \, \texttt{do} \, \langle Stmt \rangle \, \texttt{od} \\
 & & \mid \{ \, \langle StmtSeq \rangle \, \} \\
\langle StmtSeq \rangle & ::= & \epsilon \mid \langle Contract \rangle \, (; \langle Contract \rangle)^* \\
\end{array}
$$

Contracts (inline verification annotations).

$$\langle Contract \rangle \quad ::= \quad \langle Stmt \rangle \, [\, \{ \, \langle Spec \rangle \, \} \,]$$

Programs

$$\langle Program \rangle \quad ::= \quad \langle StmtSeq \rangle$$

2 Semantics

2.1 Important notions for formalizing `SelVeri`

2.1.1 Types

Let **Type** be the set of all possible types. For the current implementation of `SelVeri`, an element of **Type** can be either **Int**, **Float** or a **List** of any dimension, including one of the former two, which can be interpreted as a dependent type. More formally:

- **Int** : **Type**
- **Float** : **Type**
- **List** : $(\Pi t : \mathbf{Type} \,.\, (\Pi n : \mathbb{N}^+ \,.\, \mathbf{Type}))$

2.1.2 Immediates

Let **Imm** be the set of syntactic immediate values supported by `SelVeri`, e.g. 5 or 3.14. For each type, we need an evaluation function that translates syntactic immediates into the corresponding semantic values.

$$\mathcal{I} : \mathbf{Imm} \cap \mathbf{Int} \to \mathbb{Z}$$

$$\mathcal{F} : \mathbf{Imm} \cap \mathbf{Float} \to \mathbb{R}$$

$$\mathcal{L}_{\mathbf{List}[t,\,n]} : \mathbf{Imm} \cap \mathbf{List}[t, n] \to A^n$$

where A is the codomain of $\mathcal{L}_t$. The reader should notice that:

- $\mathcal{L}_{\mathbf{Int}} = \mathcal{I}$
- $\mathcal{L}_{\mathbf{Float}} = \mathcal{F}$

For simplicity, we also define $\mathcal{L} := \bigcup_{t \in \mathbf{Type}} \mathcal{L}_t$

2.1.3 Scope

Let **Scope** be the set of all scopes and let **Var** be the infinite set of all syntactic variables. Each scope is a representation of 'known' variables in a given block of statements including the variables declared until program execution reaches the corresponding block. Formally,

$$\forall s \in \mathbf{Scope} \,.\, s : \mathbf{Var} \cup \mathbf{Imm} \to \mathbf{Type}$$

Given any scope, s, immediates are mapped to the corresponding types; e.g. $s(5) = \mathbf{Int}$, , as expected.

Initially, each scope starts its lifetime as an empty scope, denoted as $\tilde{s}$,

$$\forall x \in \mathbf{Var} \,.\, \tilde{s}(x) = \mathbf{Void}$$

where **Void** is the empty type.

Moreover, each scope comes along with a (possibly empty) `parent_scope`, which is the scope of the program block that 'called' the current block. In the `SelVeri` implementation, a variable is searched in the `parent_scope`, if it cannot be found in the current scope.

2.1.4 Program states

Let **State** be the set of all possible program states and **Val** [1] be the set of all semantic values. Each state is in fact a map from **Var** and **Scope** to **Val**, representing the value of a given variable in the specified scope. Formally,

$$\forall \sigma \in \mathbf{State} \,.\, \sigma : \mathbf{Var} \times \mathbf{Scope} \to \mathbf{Val}$$

Each program starts with an empty state, $\tilde{\sigma}$ in which each declared variable has the default value of this type, e.g. 0 for **Int**, 0.0 for **Float** and [0, 0, 0] for **List[Int, 3]**. If the specific type is unknown in a given context, the default value of the corresponding type is denoted as **0**.

Additionally, each program state carries an `error_flag` that is set if and only if an error occurs during the execution of the program. A cause for this can be using a variable before declaring it. In order to embed this misusage as an error into our semantics, we add the following rule to our program states:

$$\forall \sigma \in \mathbf{State} \,.\, \sigma(x, s) = \bot \quad \text{if } s(x) = \mathbf{Void}.$$

Before moving to the next section, reader should be aware of an abuse of notation we use for convenient representation of erroneous states:

- σ represents a program state without any errors.
- $\hat{\sigma}$ represents a program state with an error.
- $\underline{\sigma}$ represents any program state (may be error-free or erroneous).

[1] The reader can notice that ran$ran\mathcal{I}$, ran$ran\mathcal{F}$ and $ran\mathcal{L}$ are all subsets of **Val**.

2.2 Semantics of arithmetic expressions

Let **AExp** be the set of all arithmetic expressions. We define the arithmetic valuation function $\mathcal{A}$, representing the semantics of the arithmetic expressions in `SelVeri`.

$$\mathcal{A} : \mathbf{AExp} \times \mathbf{State} \times \mathbf{Scope} \to \mathbf{Val}$$

$$\mathcal{A}(x, \sigma, s) = \mathcal{I}(x) \qquad \text{if } x \in \mathbf{Imm} \wedge \mathcal{T}(x, s) = \mathbf{Int}$$

$$\mathcal{A}(x, \sigma, s) = \mathcal{F}(x) \qquad \text{if } x \in \mathbf{Imm} \wedge \mathcal{T}(x, s) = \mathbf{Float}$$

$$\mathcal{A}(x, \sigma, s) = \mathcal{L}_{\mathbf{List}[t,n]}(x) \qquad \text{if } x \in \mathbf{Imm} \wedge \exists t \in \mathbf{Type} \cdot s(x) = \mathbf{List}[t] \wedge \mathcal{A}(\texttt{len(}x\texttt{)}, \sigma, s) = n$$

$$\mathcal{A}(x, \sigma, s) = \sigma(x, s) \qquad \text{if } x \in \mathbf{Var}$$

$$\mathcal{A}(a_1 + a_2, \sigma, s) = \begin{cases} \mathcal{A}(a_1, \sigma, s) + \mathcal{A}(a_2, \sigma, s) & \text{if } \mathcal{A}(a_1, \sigma, s) \neq \bot \wedge \mathcal{A}(a_2, \sigma, s) \neq \bot \\ \bot & \text{if } \mathcal{A}(a_1, \sigma, s) = \bot \vee \mathcal{A}(a_2, \sigma, s) = \bot \end{cases}$$

$$\mathcal{A}(a_1 * a_2, \sigma, s) = \begin{cases} \mathcal{A}(a_1, \sigma, s) \cdot \mathcal{A}(a_2, \sigma, s) & \text{if } \mathcal{A}(a_1, \sigma, s) \neq \bot \wedge \mathcal{A}(a_2, \sigma, s) \neq \bot \\ \bot & \text{if } \mathcal{A}(a_1, \sigma, s) = \bot \vee \mathcal{A}(a_2, \sigma, s) = \bot \end{cases}$$

$$\mathcal{A}(a_1 - a_2, \sigma, s) = \begin{cases} \mathcal{A}(a_1, \sigma, s) - \mathcal{A}(a_2, \sigma, s) & \text{if } \mathcal{A}(a_1, \sigma, s) \neq \bot \wedge \mathcal{A}(a_2, \sigma, s) \neq \bot \\ \bot & \text{if } \mathcal{A}(a_1, \sigma, s) = \bot \vee \mathcal{A}(a_2, \sigma, s) = \bot \end{cases}$$

$$\mathcal{A}(a_1 / a_2, \sigma, s) = \begin{cases} \lfloor \mathcal{A}(a_1, \sigma, s) / \mathcal{A}(a_2, \sigma, s) \rfloor & \text{if } \mathcal{A}(a_2, \sigma, s) \neq 0 \wedge (\mathcal{A}(a_1, \sigma, s) \neq \bot \wedge \mathcal{A}(a_2, \sigma, s) \neq \bot) \\ \bot & \text{if } \mathcal{A}(a_2, \sigma, s) = 0 \vee (\mathcal{A}(a_1, \sigma, s) = \bot \vee \mathcal{A}(a_2, \sigma, s) = \bot) \end{cases}$$

Table 1: The semantics of arithmetic expressions

Although not explicitly stated in the table above, given any binary operation, operators are required to be of the same type (except for the cases of implicit coercion [2] handled by the compiler) and can only be of *basic types* [3]. Otherwise, the result of the evaluation is $\bot$. The same rules apply to the next section (boolean expressions) as well.

[2] An example is multiplying an integer with a float. In such cases, compiler converts the integer to a float with the same value. Conversion of integers to floats is the only possibility for an implicit coercion, as for the current implementation of `SelVeri`.

[3] The only basic types are **Int** and **Float** for the current implementation of `SelVeri`.

2.3 Semantics of boolean expressions

Let **BExp** be the set of all boolean expressions. We define the arithmetic valuation function $\mathcal{B}$, representing the semantics of the boolean expressions in `SelVeri`.

$$\mathcal{B} : \mathbf{BExp} \times \mathbf{State} \times \mathbf{Scope} \to \{0, 1, \bot\}$$

$$
\begin{aligned}
\mathcal{B}(\texttt{true}, \sigma, s) &= 1 \\
\mathcal{B}(\texttt{false}, \sigma, s) &= 0 \\
\mathcal{B}(a, \sigma, s)\ ^{4} &= \begin{cases} 1 & \text{if } \mathcal{A}(a, \sigma, s) \neq \mathbf{0} \\ 0 & \text{if } \mathcal{A}(a, \sigma, s) = \mathbf{0} \\ \bot & \text{if } \mathcal{A}(a, \sigma, s) = \bot \end{cases} \\
\mathcal{B}(a_1 = a_2, \sigma, s) &= \begin{cases} 1 & \text{if } \mathcal{A}(a_1, \sigma, s) = \mathcal{A}(a_2, \sigma, s) \\ 0 & \text{if } \mathcal{A}(a_1, \sigma, s) \neq \mathcal{A}(a_2, \sigma, s) \\ \bot & \text{if } \mathcal{A}(a_1, \sigma, s) = \bot \vee \mathcal{A}(a_2, \sigma, s) = \bot \end{cases} \\
\mathcal{B}(a_1 \leq a_2, \sigma, s) &= \begin{cases} 1 & \text{if } \mathcal{A}(a_1, \sigma, s) \leq \mathcal{A}(a_2, \sigma, s) \\ 0 & \text{if } \mathcal{A}(a_1, \sigma, s) > \mathcal{A}(a_2, \sigma, s) \\ \bot & \text{if } \mathcal{A}(a_1, \sigma, s) = \bot \vee \mathcal{A}(a_2, \sigma, s) = \bot \end{cases} \\
\mathcal{B}(\texttt{not } b, \sigma, s) &= \begin{cases} 1 & \text{if } \mathcal{B}(b, \sigma, s) = 0 \\ 0 & \text{if } \mathcal{B}(b, \sigma, s) = 1 \\ \bot & \text{if } \mathcal{B}(b, \sigma, s) = \bot \end{cases} \\
\mathcal{B}(b_1 \texttt{ and } b_2, \sigma, s) &= \begin{cases} 1 & \text{if } \mathcal{B}(b_1, \sigma, s) = 1 \text{ and } \mathcal{B}(b_2, \sigma, s) = 1 \\ 0 & \text{if } \mathcal{B}(b_1, \sigma, s) = 0 \text{ or } \mathcal{B}(b_2, \sigma, s) = 0 \\ \bot & \text{if } \mathcal{B}(b_1, \sigma, s) = \bot \text{ or } \mathcal{B}(b_2, \sigma, s) = \bot \end{cases}
\end{aligned}
$$

Table 2: Semantics of boolean expressions

[4] To have this valuation rule, we need $\mathbf{AExp} \subseteq \mathbf{BExp}$ which is the case for `SelVeri`. What this rule means is, every arithmetic expression can also be interpreted as a boolean expression depending on the context and its truth value is determined by an equality to zero check, similar to the `C` programming language.

2.4 Structural operational semantics of the statements

$[\text{decl}_{\text{os}}]$	$\langle x : t, \sigma, s\rangle \to s[x \mapsto t] \quad \text{if } t \in \mathbf{Type} \wedge s(x) = \mathbf{Void}$
$[\text{decl}^{\perp}_{\text{os}}]$	$\langle x : t, \sigma, s\rangle \to \hat{\sigma} \quad \text{if } t \notin \mathbf{Type} \vee s(x) \neq \mathbf{Void}$
$[\text{ass}_{\text{os}}]$	$\langle x := a, \sigma, s\rangle \to \sigma[x \mapsto \mathcal{A}(a, \sigma, s)] \quad \text{if } \mathcal{A}(a, \sigma, s) \neq \perp \wedge \mathcal{A}(a, \sigma, s) \in ran\mathcal{L}_{s(x)}$
$[\text{ass}^{\perp}_{\text{os}}]$	$\langle x := a, \sigma, s\rangle \to \hat{\sigma} \quad \text{if } \mathcal{A}(a, \sigma, s) = \perp \vee \mathcal{A}(a, \sigma, s) \notin ran\mathcal{L}_{s(x)}$
$[\text{noop}_{\text{os}}]$	$\langle \texttt{noop}, \sigma, s\rangle \to \sigma$
$[\text{comp}^1_{\text{os}}]$	$\frac{\langle S_1,\sigma,s\rangle \to \langle S_1',\underline{\sigma}',s'\rangle}{\langle S_1;S_2,\sigma,s\rangle \to \langle S_1';S_2,\underline{\sigma}',s'\rangle}$
$[\text{comp}^2_{\text{os}}]$	$\frac{\langle S_1,\sigma,s\rangle \to \underline{\sigma}'}{\langle S_1;S_2,\sigma,s\rangle \to \langle S_2,\underline{\sigma}',s\rangle}$
$[\text{if}^1_{\text{os}}]$	$\langle \texttt{if } b \texttt{ then } S_1 \texttt{ else } S_2 \texttt{ fi}^{5}, \sigma, s\rangle \to \langle S_1, \sigma, s\rangle \quad \text{if } \mathcal{B}(b, \sigma, s) = 1$
$[\text{if}^0_{\text{os}}]$	$\langle \texttt{if } b \texttt{ then } S_1 \texttt{ else } S_2 \texttt{ fi}, \sigma, s\rangle \to \langle S_2, \sigma, s\rangle \quad \text{if } \mathcal{B}(b, \sigma, s) = 0$
$[\text{if}^{\perp}_{\text{os}}]$	$\langle \texttt{if } b \texttt{ then } S_1 \texttt{ else } S_2 \texttt{ fi}, \sigma, s\rangle \to \hat{\sigma} \quad \text{if } \mathcal{B}(b, \sigma, s) = \perp$
$[\text{while}^1_{\text{os}}]$	$\langle \texttt{while } b \texttt{ do } S \texttt{ od}, \sigma, s\rangle \to \langle S; \texttt{while } b \texttt{ do } S \texttt{ od}, \sigma, s\rangle \quad \text{if } \mathcal{B}(b, \sigma, s) = 1$
$[\text{while}^0_{\text{os}}]$	$\langle \texttt{while } b \texttt{ do } S \texttt{ od}, \sigma, s\rangle \to \sigma \quad \text{if } \mathcal{B}(b, \sigma, s) = 0$
$[\text{while}^{\perp}_{\text{os}}]$	$\langle \texttt{while } b \texttt{ do } S \texttt{ od}, \sigma, s\rangle \to \hat{\sigma} \quad \text{if } \mathcal{B}(b, \sigma, s) = \perp$
$[\text{err}_{\text{os}}]$	$\langle S, \hat{\sigma}, s\rangle \to \hat{\sigma}$

Table 3: Small-step operational semantics for statements

[5] For each if-rule, an `if`-statement without the `else`-part, namely a statement of the form "`if` b `then` S `fi`" can be treated as "`if` b `then` S `else noop fi`"